

SafeAid — Frontend Brief

For: Ibrahim Ashiah Ajoke (Asaiah)

Project: SafeAid Emergency Support Platform

Stack: Next.js 15 (App Router), TypeScript, Tailwind CSS

Backend developer: Emmanuel (backend is done — API is live on Render)

What SafeAid Is

SafeAid is an AI-powered emergency routing platform for vulnerable people in Nigeria.

- **Basic phone users** (trafficking survivors, abuse victims) send an SMS to a SafeAid number. The backend AI analyses their message, asks what kind of help they need, then texts back the nearest verified shelter, NGO, or emergency contact.
- **The dashboard you are building** is used by caseworkers and NGO staff to monitor incoming alerts in real time, see what help was recommended, read the conversation thread, and mark cases resolved.

You are building the caseworker dashboard — not something survivors interact with directly.

Your Role

You own everything inside the `/frontend` folder. Emmanuel owns `/backend`.

You do not need to touch the backend. Emmanuel has already built and deployed the API. You will connect to it in the final integration phase — but for now, build everything with hardcoded mock data first, then swap to real API calls at the end.

Folder Structure

Work only inside `/frontend`. This is the structure you will build:

- **app/**
 - `layout.tsx` (Main layout: sidebar + main area)
 - `globals.css` (Base Tailwind styles)
 - `page.tsx` (Dashboard home / cases list)
 - **cases/**
 - *id*
 - `page.tsx` (Case detail + SMS conversation thread)

- **help-centers/**
 - **page.tsx** (Browsable directory of NGOs + shelters)
- **app/components/**
 - **Sidebar.tsx** (Left nav: SafeAid logo, links, footer)
 - **StatsHeader.tsx** (3 metric cards at top of dashboard)
 - **CaseCard.tsx** (Individual case card on dashboard)
 - **RiskBadge.tsx** (HIGH / MEDIUM / LOW coloured pill)
 - **ResourceBadge.tsx** (shelter / food / legal / medical / jobs pill)
 - **MessageBubble.tsx** (SMS conversation bubble)
 - **VoiceTranscriptPanel.tsx** (Simulated voice call transcript)

Start with layout.tsx -> Sidebar.tsx -> page.tsx -> components. Do not jump ahead.

Design Direction

Feeling: Calm, clinical, trustworthy. Think a well-designed hospital management system — not a startup landing page. Caseworkers are handling stressful, high-stakes situations. The UI must reduce noise, not add to it.

Layout: Dark sidebar on the left, white/light main content area on the right.

Most important visual rule: Risk level is the loudest signal on every screen. HIGH cases must be immediately visible. Everything else is subordinate to that.

Colours to use

- **Sidebar background:** #0F172A (slate-900)
- **Main background:** #F8FAFC (slate-50)
- **Card background:** #FFFFFF
- **Border:** #E2E8F0 (slate-200)
- **Brand / links:** #F97316 (orange-500)
- **Brand hover:** #EA580C (orange-600)
- **Brand light:** #FFF7ED (orange-50)
- **Text primary:** #0F172A
- **Text secondary:** #64748B (slate-500)
- **Text on sidebar:** #F8FAFC
- **Active nav text:** #F97316 (orange-400 on dark bg)
- **Active nav bg:** #431407 (orange-950 — subtle on dark sidebar)

AI-generated content

Any text generated by the AI must have a visible "AI suggestion" label above it in a light blue block.

Icons

Use **Lucide React** — outline style only, 16px inline, 20px for nav.

- LayoutDashboard (Dashboard nav link)
- AlertTriangle (High risk indicator)
- CheckCircle2 (Resolved cases)
- MessageSquare (SMS / message thread)
- Phone (Voice panel)
- MapPin (Help center location)
- Clock (Timestamp)
- Building2 (Help centers nav link)

What to Build (Your Units)

Build these in order. Do not skip ahead. Finish one before starting the next.

Phase 1 — Layout Shell

- **U4.1** — Next.js setup + layout with dark sidebar and white main area. Includes Sidebar SafeAid wordmark, nav links, and Responsible AI note.

Phase 2 — Component Library (build with hardcoded data)

- **U4.2** — RiskBadge.tsx and ResourceBadge.tsx
- **U4.3** — StatsHeader.tsx (Total Active, High Risk, Resolved Today)
- **U4.4** — CaseCard.tsx (Coloured left border, badges, AI summary, routing block, timestamp. Clickable to detail page).
- **U4.5** — app/page.tsx (Dashboard) — StatsHeader + list of 4 case cards using mock data.
- **U4.6** — app/help-centers/page.tsx — list of center cards (name, type badge, state, phone, Get Directions link).

Phase 3 — Case Detail

- **U5.1** — MessageBubble.tsx (Inbound left/grey, Outbound right/orange).
- **U5.2** — app/cases/[id]/page.tsx (Back button, badges, AI summary, routing panel, scrollable thread, "Mark Resolved" button).
- **U5.3** — VoiceTranscriptPanel.tsx (Pulsing red live dot, "Simulated for demo" label, 4-line transcript. Add to bottom of dashboard).

Phase 4 — Integration

Note: The backend API is deployed on Render at <https://safeaid.onrender.com>. Set your .env.local variable to `NEXT_PUBLIC_API_URL=https://safeaid.onrender.com`.

- **U6.1** — Replace mock cases on page.tsx with real GET /api/cases.
- **U6.2** — Replace mock messages on case detail with real GET /api/cases/:id/messages. Wire resolve button.
- **U6.3** — Replace mock centers with real GET /api/centers. Add filters.

Rules

1. **Pages fetch. Components render.** Never put `fetch()` calls inside a component.
2. **No raw phone numbers.** Display hashes as "Anonymous contact".
3. **Always label AI output.** 4. **Always label the voice panel as simulated.**
4. **One unit at a time.** Finish and verify before moving to the next.
5. **Update progress tracker** after each unit.